

HTTP Server Experiment

HTTP Server Experiment

1. HTTP Overview
2. Core Concepts
 - 2.1 HTTP Request-Response Model
 - 2.2 HTTP Methods
 - 2.3 HTTP Status Codes
 - 2.4 esp_http_client Event-Driven Model
3. Project Architecture and Flow
 - 3.1 HTTP Server
 - 3.3 Server Routing Table
4. Hardware Dependencies
5. Source Code Analysis — HTTP Server
 - 5.1 Route Handlers
 - 5.2 start_http_server() — Start Server
6. Operation Flow
 - 6.1 Build and Flash Flow
 - 6.2 Testing HTTP Server
 - 6.3 Normal Log — HTTP Server
7. Common Problems

1. HTTP Overview

HTTP is the "passport" for IoT devices to integrate into the Web ecosystem—mastering HTTP communication allows ESP32 to access any API on the internet like a browser, and also turn itself into a Web server for direct device control via webpages from phones or computers. Whether doing remote sensor data reporting, weather fetching, OTA firmware upgrades, or building smart home web control panels, HTTP is the most versatile choice. This tutorial covers the server RESTful API routine, teaching core HTTP communication skills on ESP32.

HTTP (HyperText Transfer Protocol) is the most widely used application layer protocol on the internet. It is based on TCP and uses a **request-response** model: client sends GET request, server returns status code + data.

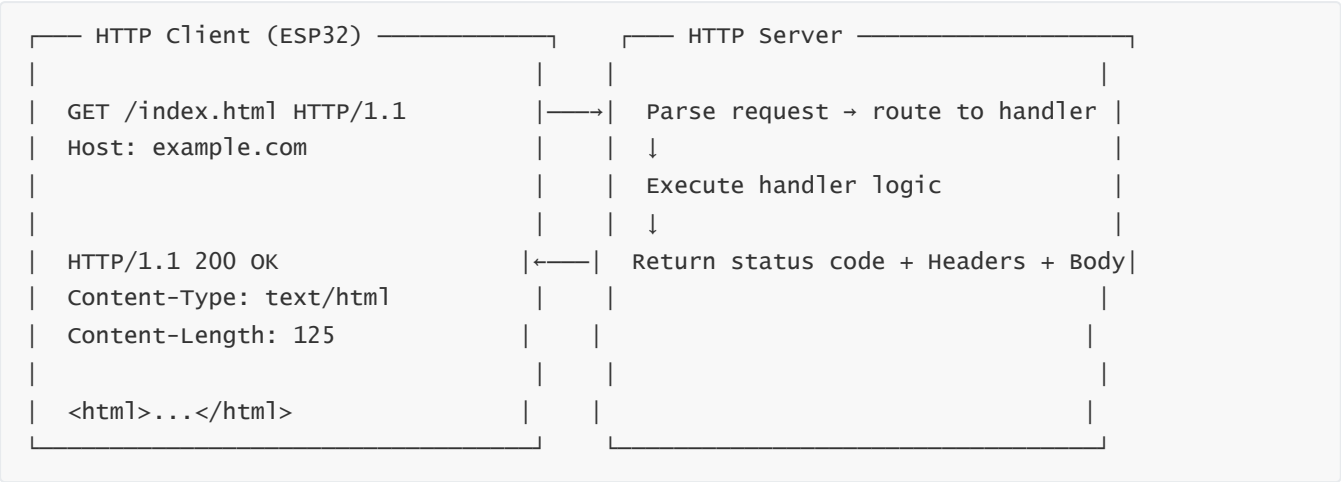
Feature	HTTP	Bare TCP/UDP
Protocol layer	Application layer (based on TCP)	Transport layer
Data format	Standard Headers + Body	Custom
Status code	200/404/500 etc. standard semantics	None
Cross-platform compatibility	Browser/curl/HTTP libraries for all languages	Need custom parsing

Feature	HTTP	Bare TCP/UDP
Applicable scenarios	Web API, file download, RESTful	Custom protocols, extreme performance scenarios

This project contains both HTTP client and server routines: the client gets webpage data via GET request; the server provides RESTful API on port 80.

2. Core Concepts

2.1 HTTP Request-Response Model



2.2 HTTP Methods

Method	Semantics	Request Body	Typical Use
GET	Get resource	None	Query data, get webpage

2.3 HTTP Status Codes

Status Code	Semantics	Description
200	OK	Request successful
206	Partial Content	Range request for partial content (OTA firmware download)
301	Moved Permanently	Permanent redirect
404	Not Found	Requested resource does not exist
500	Internal Server Error	Server internal error

2.4 esp_http_client Event-Driven Model

ESP-IDF's HTTP client uses event callback mechanism. Key events:

Event	Trigger	Callback Parameters
<code>HTTP_EVENT_ON_CONNECTED</code>	TCP connection established	—
<code>HTTP_EVENT_ON_DATA</code>	Response data chunk received	<code>data</code> + <code>data_len</code>
<code>HTTP_EVENT_ON_FINISH</code>	Request completed	—
<code>HTTP_EVENT_ERROR</code>	Request error	—

3. Project Architecture and Flow

3.1 HTTP Server

```
app_main()
└─ xTaskCreate(vHttpServerTask, stack=8KB, prio=5)
    └─ vHttpServerTask():
        │ wifici_init()                ← WiFi connect + wait for IP
        │ start_http_server()
        │ │ httpd_start(port 80)        ← Start HTTP server
        │ │ Register GET "/"            → root_handler
        │ │ Register GET "/get"         → get_handler
        └─ while(1) vTaskDelay(1000ms)  ← Keep running
```

3.3 Server Routing Table

Routing determines "access which path → execute which function → return what content":

- `GET /` → call `root_handler`, return `{"msg":"ESP32 HTTP Server is running!"}`
- `GET /get` → call `get_handler`, return `{"status":"ok","msg":"hello from esp32 http server"}`

Both have Content-Type `application/json`.

4. Hardware Dependencies

- **Hardware Connection:** No external wiring, WiFi wireless communication
- **Client required components:** `esp_http_client`, `esp_wifi`, `esp_netif`, `esp_event`, `nvs_flash`
- **Server required components:** `esp_http_server` (instead of `esp_http_client`), others same as above

5. Source Code Analysis — HTTP Server

Full source: `Source Code` → `ESP-IDF` → `5.Network_Protocol` → `6.HTTP` → `HTTP_Server`

5.1 Route Handlers

The most core concept of HTTP server is **routing**—client (browser/Postman) accesses different paths, ESP32 returns different content. Each route is a function that writes response to `req`. `httpd_resp_set_type()` sets Content-Type (set `"application/json"` for returning JSON, `"text/html"` for webpages), `httpd_resp_send()` sends the content out.

```
static esp_err_t root_handler(httpd_req_t *req)
{
    char resp[] = "{\"msg\":\"ESP32 HTTP Server is running!\"}";
    httpd_resp_set_type(req, "application/json");
    httpd_resp_send(req, resp, strlen(resp));
    ESP_LOGI(TAG, "Root / Request processed successfully");
    return ESP_OK;
}

static esp_err_t get_handler(httpd_req_t *req)
{
    char resp[] = "{\"status\":\"ok\", \"msg\":\"hello from esp32 http server!\"}";
    httpd_resp_set_type(req, "application/json");
    httpd_resp_send(req, resp, strlen(resp));
    ESP_LOGI(TAG, "GET Request processed successfully");
    return ESP_OK;
}
```

5.2 start_http_server() — Start Server

`start_http_server()` starts an HTTP server, then hangs routes one by one using `httpd_register_uri_handler()`. Each `httpd_uri_t` specifies three things: path (`.uri`), HTTP method (`.method`), handler function (`.handler`). Client accessing `http://ESP32_IP/` calls `root_handler`, accessing `/get` calls `get_handler`.

```
static void start_http_server(void)
{
    httpd_config_t config = HTTPD_DEFAULT_CONFIG();
    httpd_handle_t server = NULL;

    if (httpd_start(&server, &config) != ESP_OK) {
        ESP_LOGE(TAG, "Failed to start HTTP server!");
        return;
    }
    ESP_LOGI(TAG, "HTTP server started, port: 80");

    httpd_uri_t root_uri = {
        .uri      = "/",
        .method   = HTTP_GET,
        .handler   = root_handler,
        .user_ctx = NULL,
    };
    httpd_register_uri_handler(server, &root_uri);

    httpd_uri_t get_uri = {
```

```

        .uri      = "/get",
        .method   = HTTP_GET,
        .handler   = get_handler,
        .user_ctx = NULL,
    };
    httpd_register_uri_handler(server, &get_uri);
}

```

6. Operation Flow

6.1 Build and Flash Flow

ESP-IDF Flash Flow: See `ESP-IDF → 1. Before Development → 3. Build and Flash`.

6.2 Testing HTTP Server

1. Flash HTTP_Server to ESP32. Note the IP address printed on serial (e.g. `192.168.11.100`).
2. Open Postman, create GET request, URL input `http://192.168.11.100/` → returns `{"msg":"ESP32 HTTP Server is running!"}`
3. Same method, GET request `http://192.168.11.100/get` → returns `{"status":"ok","msg":"hello from esp32 http server"}`

6.3 Normal Log — HTTP Server

```

I (3856) HTTP-SERVER: ===== HTTP SERVER =====
I (3858) HTTP-SERVER: Waiting for WiFi connection...
I (3872) HTTP-SERVER: Got IP: 192.168.11.100
I (3874) HTTP-SERVER: WiFi connected successfully!
I (3876) HTTP-SERVER: HTTP server started, port: 80
I (5000) HTTP-SERVER: Root / Request processed successfully
I (6000) HTTP-SERVER: GET Request processed successfully

```

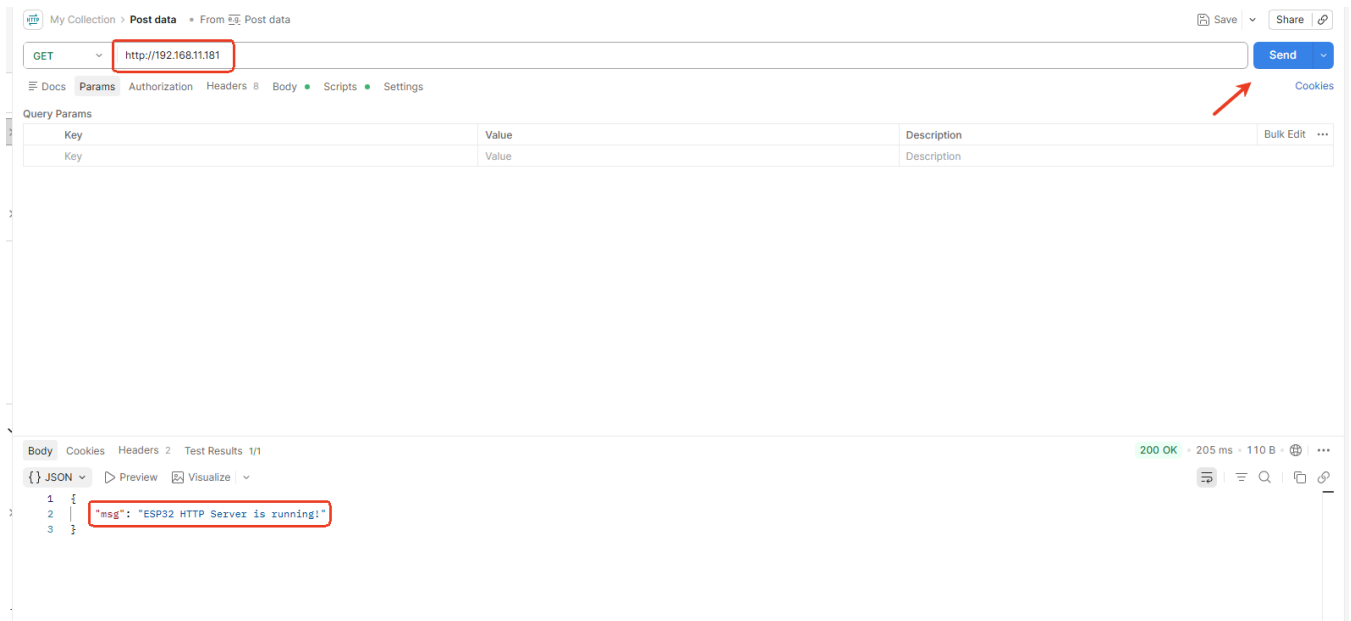
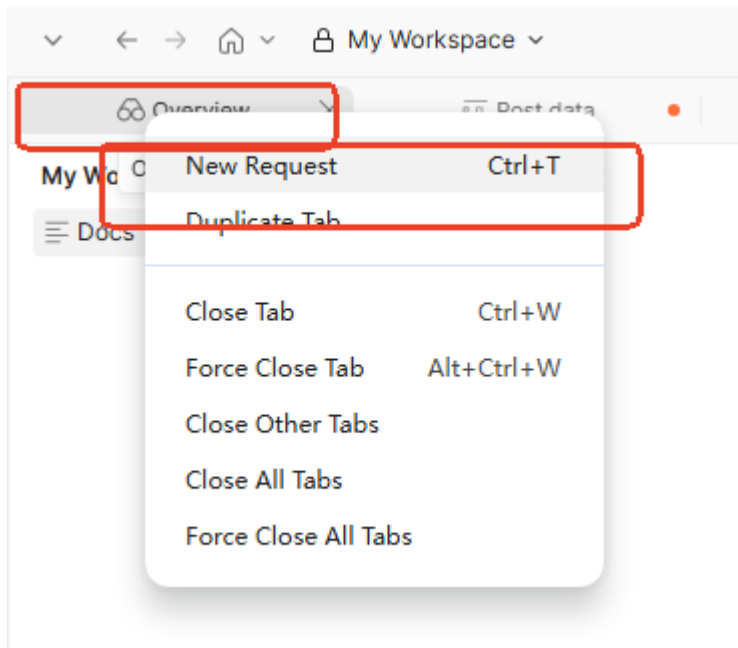
```

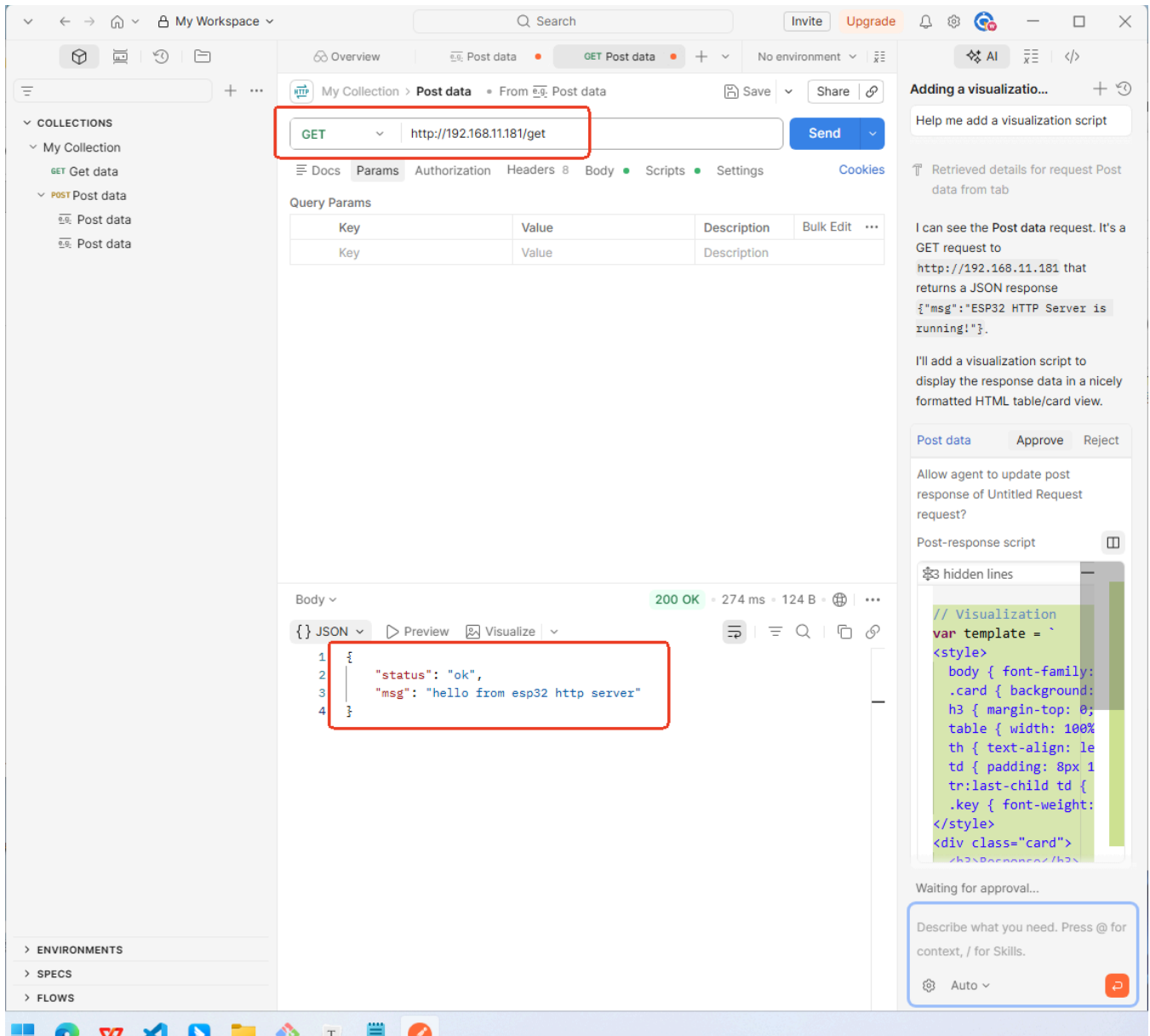
I (1717) esp_netif_handlers: sta ip: 192.168.11.181, mask: 255.255.255.0, gw: 192.168.11.1
I (1717) HTTP-SERVER: Got IP: 192.168.11.181
I (1717) HTTP-SERVER: WiFi connected successfully!
I (1717) HTTP-SERVER: HTTP server started, port: 80
I (8097) wifi:<ba-add>idx:1 (ifx:0, 80:8f:1d:20:9d:f7), tid:0, ssn:5, winSize:64
I (8107) HTTP-SERVER: GET Request processed successfully

```

Use Postman to test, create GET request, URL fill ESP32's IP and port:

- GET `http://<ESP32_IP>/` → returns `{"msg":"ESP32 HTTP Server is running!"}`
- GET `http://<ESP32_IP>/get` → returns `{"status":"ok","msg":"hello from esp32 http server"}`





7. Common Problems

Symptom	Cause	Solution
HTTP client <code>HTTP request error</code>	URL unreachable or DNS resolution failed	Ensure ESP32 can access external network; or use IP address instead of domain name
<code>esp_http_client_init()</code> returns NULL	URL format error	Check <code>HTTP_URL</code> starts with <code>http://</code>
Postman/browser accessing ESP32 no response	Not on same LAN / firewall blocked	Ensure PC and ESP32 are on same router
<code>httpd_start()</code> failed	Port 80 in use	Use port like <code>8080</code>
Content-Type incorrect causing client parsing failure	<code>httpd_resp_set_type()</code> not called	Must set <code>"application/json"</code> when returning JSON